



TAYLOR'S
UNIVERSITY
Wisdom • Integrity • Excellence

Bachelor of Computer Science (Hons)

Bachelor of Software Engineering (Hons)

Bachelor of Information Technology (Hons)

Module Code : ITS60504 (Sept 2024)

Module Name : Data Structures and Algorithms

Assignment No./Title	Group Assignment Task (30%)
Course Tutor/Lecturer	Mr. Sham Shul Shukri Mat, Dr Saliyah Kahar
Submission Date	Monday 30 th December 1159 pm (Week 14)
Student Name, ID and Signature 1. Xu Jing 0372834 <i>Jing</i> 2. Xiong ZeHao 0373416 <i>Zehao</i> 3. Zhong ZhenYuan 0371321 <i>Zhenyuan</i> 4. Xing ZiXiang 0368813 <i>Zixiang</i> 5. Wong Li En 0374089 <i>Lien</i> 6. Yong Zhen Ming 0374974 <i>Yong</i>	

Declaration *(need to be signed by students. Otherwise, the assessment will not be evaluated)*

Certify that this assignment is entirely my own work, except where I have given fully documented references to the work of others, and that the material contained in this assignment has not previously been submitted for assessment in any other formal course of study.

Marks/Grade:	Evaluated by:
Evaluator's Comments:	

Part 1- Algorithm Analysis & Development (20 marks)

The following is a list of the processing time $T(n)$ spent by an algorithm for solving a problem of size n . For each of these questions (a) to (f), analyze the terms, provide dominant term, specify the Big O complexity, and justify your answer by comparing with all other terms.

a) $3 + \log_2 n + 2n + n \log_3 27 + 4$
 $= 3 + \log_2 n + 2n + 3n + 4$

Comparing Growth Rates: $3 < 4 < \log_2 n < 2n < 3n$

Dominant term : $3n$

Big O complexity : $O(n)$

Prove:

1. $(3 + \log_2 n + 2n + 3n + 4)$ is $O(n)$
2. need $c > 0$ and $n_0 \geq 1$ such that $3 + \log_2 n + 2n + 3n + 4 \leq c * n$ for $n \geq n_0$
3. this is true for $c = 10$ and $n_0 = 2$

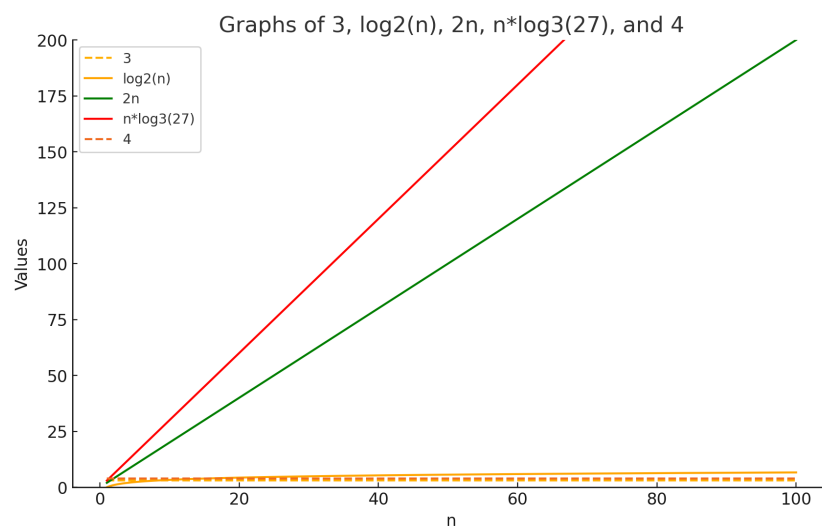


Figure 1 function of a

It can be seen from the figure 1

3 and 4 The growth rate of constant function is the slowest.

$\log_2 n$ the slowest growth

The linear function grows faster than the logarithmic function

The coefficient of $3n$ in the linear function coefficient is larger than $2n$,
so the growth rate of $3n$ is larger than $2n$.

$3n$ dominates the overall complexity

b) $\log_4 n + \log_3 n + 20 + \log_5 n + 2$

Comparing Growth Rates : $2 < \log_5 n < \log_4 n < \log_3 n$

Dominant term : $\log_3 n$

Big O complexity : $O(\log n)$

Prove:

1. $\log_4 n + \log_3 n + 20 + \log_5 n + 2$ is $O(\log n)$
2. need $c > 0$ and $n_0 \geq 1$ such that $\log_4 n + \log_3 n + 20 + \log_5 n + 2 \leq c * \log n$ for $n \geq n_0$
3. this is true for $c = 80$ and $n_0 = 2$

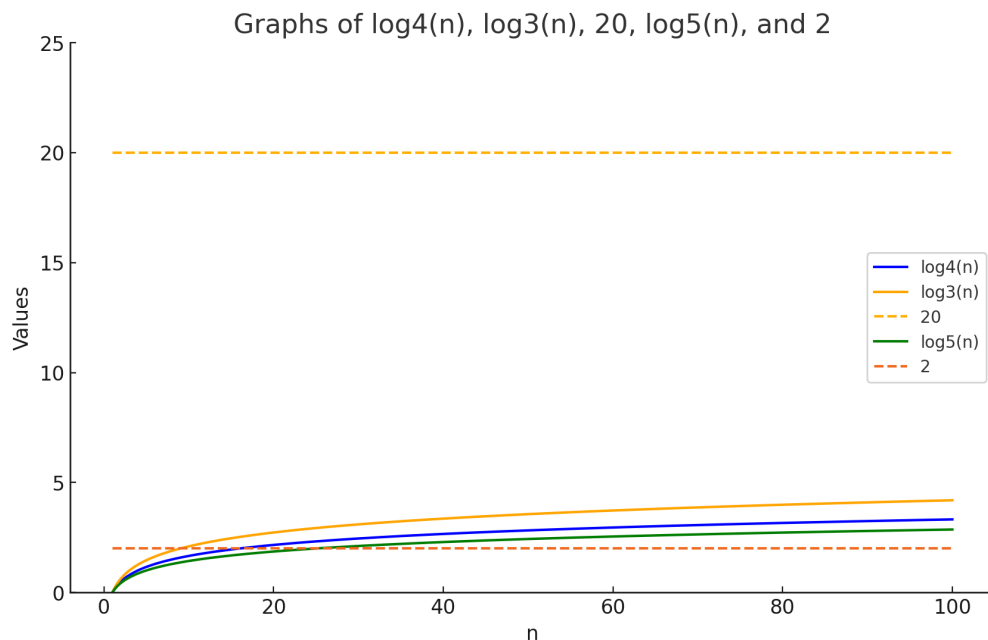


Figure 2 function of b

It can be seen from the figure 2,

2 and 20 The growth rate of constant function is the slowest.

$\log_3 n$ and $\log_4 n$ and $\log_5 n$ Both are logarithmic functions.

Because the logarithmic functions only differ in their bases, by comparing the bases we can conclude $\log_3 n$ The growth rate is the highest, followed by $\log_4 n$, and finally $\log_5 n$.

So $\log_3 n$ dominates the complexity.

$$\begin{aligned} \text{c) } & \log_2 n + n \log_2 64 + 2n + n \log_3 n \\ & = \log_2 n + 6n + 2n + n \log_3 n \end{aligned}$$

Comparing Growth Rates : $\log_2 n < 2n < 6n < n \log_3 n$

Dominant term : $n \log_3 n$

Big O complexity : $O(n \log n)$

Prove:

1. $\log_2 n + 6n + 2n + n \log_3 n$ is $O(n \log n)$
2. need $c > 0$ and $n_0 \geq 1$ such that $\log_2 n + 6n + 2n + n \log_3 n \leq c * n \log n$ for $n \geq n_0$
3. this is true for $c = 10$ and $n_0 = 10$

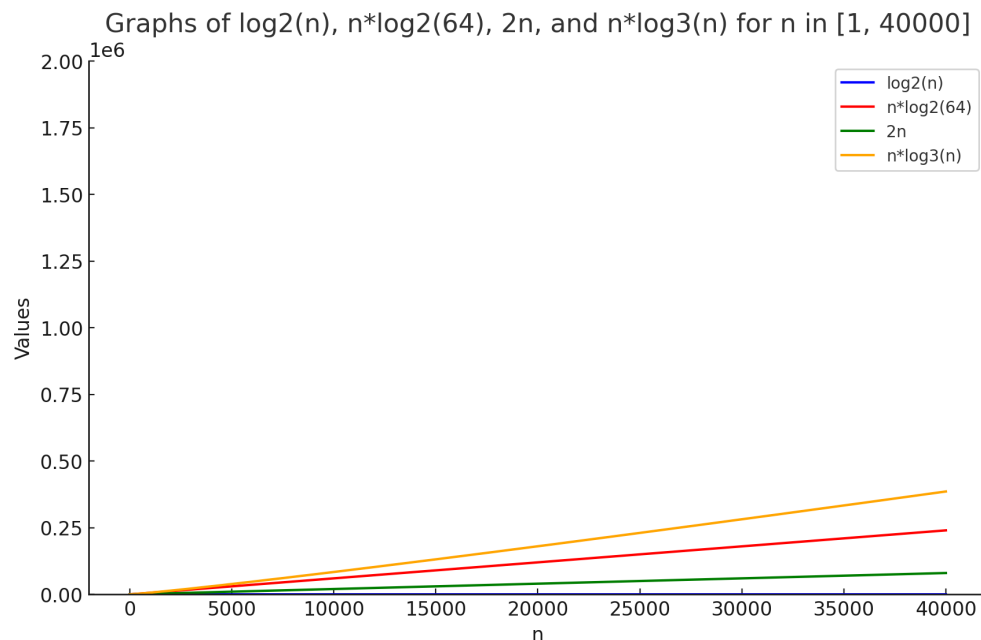


Figure 3 function of c

From the figure 3, we can see that there are logarithmic functions $\log_2 n$, linear functions $6n$ and $2n$, and logarithmic functions $n \log_3 n$.

It can be seen that the growth rate of $6n$ is faster than $n \log_3 n$ at the beginning, but the growth rate of $n \log_3 n$ is gradually approaching $6n$ and exceeding $6n$. Therefore, the function $n \log_3 n$ dominates the complexity.

d) $n \log_4 n + n(n + n^3) + 5n + 6$

Comparing Growth Rates : $6 < n \log_4 n < 5n < n^2 < n^4$

Dominant term : n^4

Big O complexity : $O(n^4)$

Prove:

1. $n \log_4 n + n(n + n^3) + 5n + 6$ is $O(n^4)$
2. need $c > 0$ and $n_0 \geq 1$ such that $n \log_4 n + n(n + n^3) + 5n + 6 \leq c * n^4$ for $n \geq n_0$
3. this is true for $c = 4$ and $n_0 = 4$

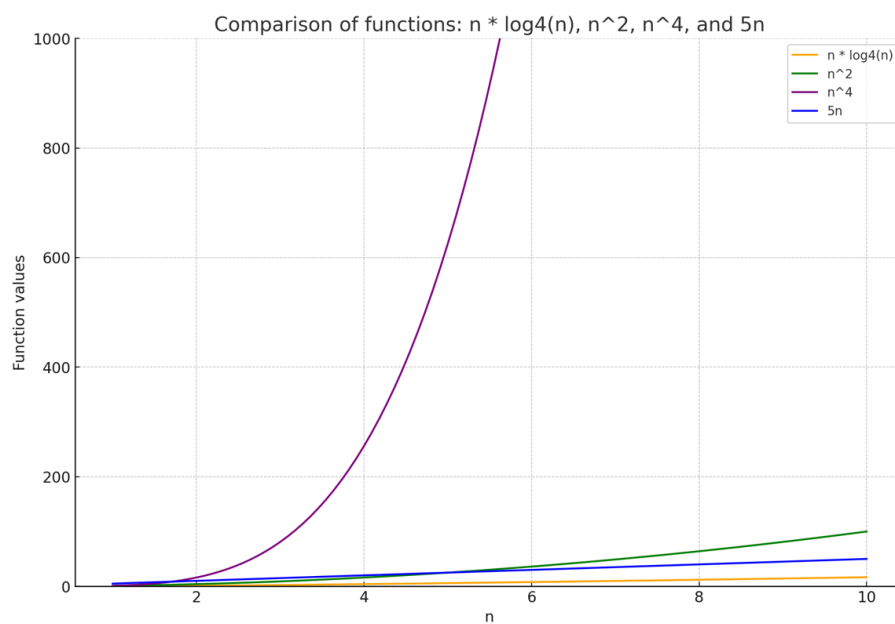


Figure 4 function of d

n^4 is the fastest growing, well ahead of the other functions.

n^2 and $n \log_4 n$ are close at the beginning, but as n increases, n^2 grows faster.

$3n$, $2n$, and $5n$ all behave linearly, but $5n$ has a larger coefficient, so it rises the fastest of the three functions.

The constant function 8 is the slowest of all functions and remains fixed.

e) $2^n + n^4 + 2n^5 + n\log_2 n + 3^n$

Comparing Growth Rates : $\log_2 n < n^4 < n^5 < 2^n < 3^n$

Dominant term : 3^n

Big O complexity : $O(3^n)$

Prove:

1. $2^n + n^4 + 2n^5 + n\log_2 n + 3^n$ is $O(3^n)$
2. need $c > 0$ and $n_0 \geq 1$ such that $2^n + n^4 + 2n^5 + n\log_2 n + 3^n \leq c * 3^n$ for $n \geq n_0$
3. this is true for $c = 5$ and $n_0 = 10$

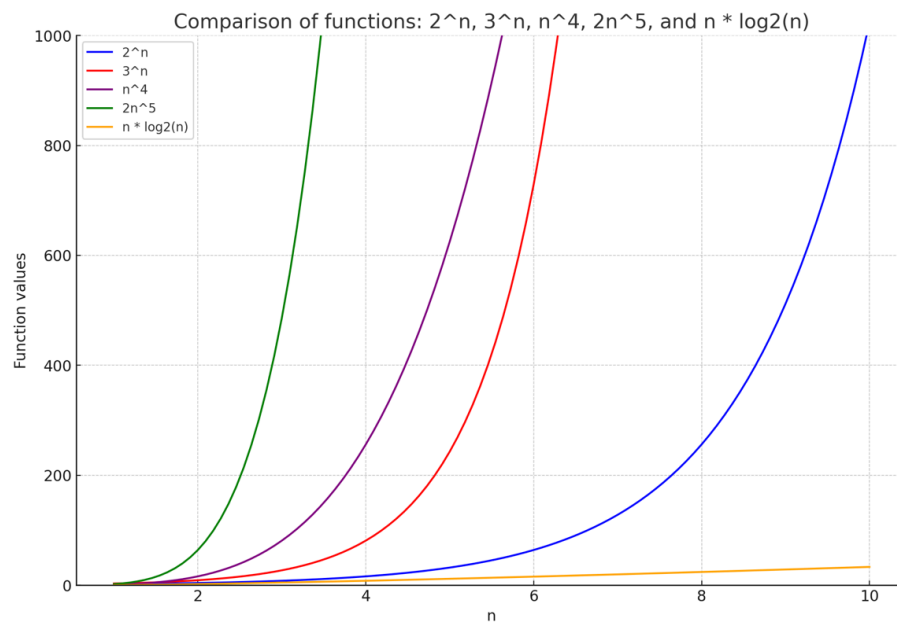


Figure 5 function of e

$n\log_2 n$ and n^4 dominate at the beginning, but as n increases, they are overtaken by higher order functions and exponential functions.

$2n^5$ grows beyond n^4 but still lags behind the exponential function.

2^n and 3^n grow exponentially with n , with 3^n growing the fastest, dominating the overall complexity.

f) $n^4 + n(n^2 + 2n + \log_5 n + 1)$

Comparing Growth Rates : $n < n \log_5 n < 2n^2 < n^3 < n^4$

Dominant term : n^4

Big O complexity : $O(n^4)$

Prove:

1. $n^4 + n(n^2 + 2n + \log_5 n + 1)$ is $O(n^4)$
2. need $c > 0$ and $n_0 \geq 1$ such that $n^4 + n(n^2 + 2n + \log_5 n + 1) \leq c * n^4$ for $n \geq n_0$
3. this is true for $c = 5$ and $n_0 = 2$

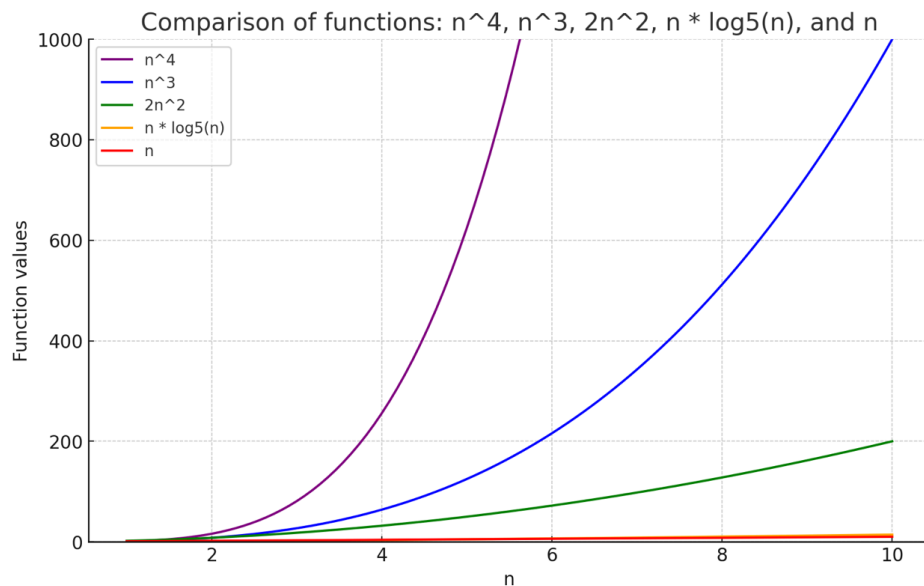


Figure 6 function of f

n^4 is the fastest growing, with a trend that far exceeds that of the other functions.

n^3 is also growing fast, second fastest after n^4 .

$2n^2$ and $n \log_5 n$ grows relatively slowly, but still faster than the linear function n .

n grows the slowest, showing linear growth.

The following code indentation may not be properly formatted. For each of these questions (g) and (h), analyze the code, provide the dominant term and the Big O complexity. Justify how you arrive at that conclusion.

g)

```
void process(int n) {  
    int tot = 0;  
    for (int a = 1; a < n; a++)  
        tot = tot + n;  
        for (int b = 1; b < n; b++)  
            for (int c = 1; c < n; c++)  
                tot = tot + n;  
    for (int d = 1; d < n; d++) {  
        n = n/2;  
        tot = tot + n;  
    }  
    for (int e = 1; e < n; e++)  
        tot = tot + n;  
}
```

The given problem comes in the form of the following code, where one must identify the dominant term and Big O Complexity of the illustrated code. This involves describing how the loops found in the code contribute to the overall computational complexity. Below is the original code provided by the question:

```
void process(int n) {  
    int tot = 0;  
    for (int a = 1; a < n; a++)  
        tot = tot + n;  
    for (int b = 1; b < n; b++)  
        for (int c = 1; c < n; c++)  
            tot = tot + n;  
    for (int d = 1; d < n; d++) {  
        n = n / 2;  
        tot = tot + n;  
    }  
    for (int e = 1; e < n; e++)  
        tot = tot + n;  
}
```

Figure 7 shows the original code provide by question

Step 1: Simplify and reformat the code

From the code above we can simplify and reformat it so that the code can be seen clearly. The simplified codes are shown below.

```
void process(int n) {  
    int tot = 0;  
  
    // First loop  
    for (int a = 1; a < n; a++)  
        tot = tot + n;  
  
    // Second nested loop  
    for (int b = 1; b < n; b++)  
        for (int c = 1; c < n; c++)  
            tot = tot + n;  
  
    // Third loop  
    for (int d = 1; d < n; d++) {  
        n = n / 2;  
        tot = tot + n;  
    }  
  
    // Fourth loop  
    for (int e = 1; e < n; e++)  
        tot = tot + n;  
}
```

Figure 8 shows the reformatted code

Step 2: Analyses the code

First Loop

```
// First loop  
for (int a = 1; a < n; a++)  
    tot = tot + n;
```

Figure 9 shows the code for First Loop

This is a simple for loop that runs from 1 to n-1

Inside the loop, a constant amount of work is done

Thus,

Time complexity for this loop: **n**

Second Nested Loop

```
// Second nested loop
for (int b = 1; b < n; b++)
    for (int c = 1; c < n; c++)
        tot = tot + n;
```

Figure 10 shows the code for Second Nested Loop

The outer loop (b) runs from 1 to n-1

The inner loop (c) also runs from 1 to n-1

So, this is a nested loop where both the outer and inner loops each run n times.

Inside the inner loop, the work done is constant (adding n to tot).

Thus,

Time complexity for this nested loop: n^2

Third Loop

```
// Third loop
for (int d = 1; d < n; d++) {
    n = n / 2;
    tot = tot + n;
}
```

Figure 11 shows the code for Third Loop

This is a for loop that runs from 1 to n-1.

However, inside the loop, the value of n is halved ($n = n / 2$), meaning that n is being reduced after each iteration.

Initially, n is large, but after each iteration, n gets smaller. Specifically, n is halved after each iteration, so the number of iterations reduces logarithmically.

The work done inside the loop (adding n to tot) takes constant time.

Thus,

Time complexity for this loop: $\log n$ (because n is halved each time)

Fourth Loop

```
// Fourth loop
for (int e = 1; e < n; e++)
    tot = tot + n;
}
```

Figure 12 shows the code for Fourth Loop

Similar to the first loop, this loop runs from 1 to $n-1$ (i.e., n iterations).

Inside the loop, a constant amount of work is done (adding n to tot).

Thus,

Time complexity for this loop: **n**

Step 3: Combining and Compare All the Complexities

Contribution from each section:

First loop: n

Second set of nested loops: n^2

Third loop: $\log n$

Fourth cycle: n

Total Complexity: The total time complexity is the sum of these individual complexities. Since the Big O notation represents the upper bound, we only consider the highest order term:

$n + n^2 + \log n + n$

The dominant term here is n^2 , since it grows faster than n and $\log n$.

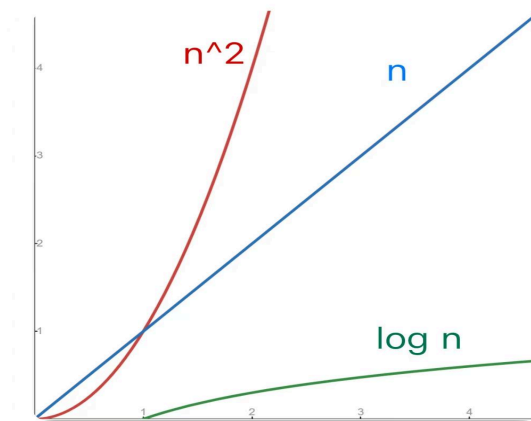


Figure 13 function of g

Thus,

Dominant term: n^2

Big O complexity: $O(n^2)$

h)

The given problem comes in the form of the following code, where one must identify the dominant term and Big O Complexity of the illustrated code. This involves describing how the loops found in the code contribute to the overall computational complexity. Below is the original code provided by the question:

```
void process(int n) {
    int tot = 0;
    for (int a = 1; a < n; a++)
        for (int b = 1; b < n; b++)
            tot = tot + n;
    for (int c = 1; c < n; c++) {
        tot = tot + n;
        for (int d = 1; d < n; d++)
            tot = tot + n;
    }
    for (int e = 1; e < n; e++)
```

```

        tot = tot + n;
    }
}

```

Step 1: Simplify and reformat the code

From the code above we can simplify and reformat it so that the code can be seen clearly. The simplified codes are shown below.

```

void process(int n) {
    int tot = 0;

    // Outer Loop A
    for (int a = 1; a < n; a++) {
        // Nested Loop B inside A
        for (int b = 1; b < n; b++) {
            tot = tot + n;
        }
    }

    // Outer Loop C
    for (int c = 1; c < n; c++) {
        tot = tot + n;
        // Nested loop for d inside c
        for (int d = 1; d < n; d++) {
            tot = tot + n;
        }
    }

    //Loop E
    for (int e = 1; e < n; e++) {
        tot = tot + n;
    }
}

```

Figure 14 shows the reformatted code

Step 2: Identify Expression Section-by-Section

Loop A and Loop B

```

for (int a = 1; a < n; a++) {

    for (int b = 1; b < n; b++) {

        tot = tot + n;

    }

}

```

Arithmetic Expression of Loop A

Since $a = 1$ and $a < n$, and n is a constant value. So, as n increases from 1 to infinity, the amount of time needed to execute this statement increases linearly. Each time you increment n , the amount of additional time needed from the previous remains constant. Thus, the loop runs $n - 1$ times

Arithmetic Expression of Loop B

For each iteration of the outer loop (a), the inner loop runs with $b = 1$ to $b < n$. Inside the inner loop, the operation $tot = tot + n$; is a constant-time operation. Therefore, the inner loop runs $n-1$ times.

Total Operations for this Loop:

Since b is nested inside a , the total number of operations is the product of the outer and inner loop iterations:

$$\begin{aligned} &(n-1) \times (n-1) \\ &= (n-1)^2 \end{aligned}$$

Expanding the expression:

$$(n-1)^2$$

$$\text{Total Operations} = n^2 - 2n + 1$$

Loop C and Loop D

```
for (int c = 1; c < n; c++) {  
  
    tot = tot + n;  
  
    for (int d = 1; d < n; d++)  
  
        tot = tot + n;  
  
}
```

Arithmetic Expression of Loop C

Since $c = 1$ and $c < n$, and n is a constant value. Inside the loop c , the statement $tot = tot + n$ is executed once, $n-1$ operations. Thus, the loop runs $n - 1$ times

Arithmetic Expression of Loop D

For each iteration of the outer loop c , the inner loop runs $n-1$ times. The inner loop d also runs from 1 to $n-1$, so it iterates $n-1$ times for each iteration of c . Inside the inner loop, the operation $tot = tot + n$; is a constant-time operation. Since the outer loop itself runs $n-1$ times. Thus, the total number of operations for the inner loop is $n - 1 \times n - 1 = (n - 1)^2$

Total Operations for this Loop:

Since Loop D is nested inside Loop C, the total number of operations is the product of the outer and inner loop iterations:

$$\begin{aligned} & (n-1) \times (n-1) \\ & = (n-1)^2 \end{aligned}$$

Expanding the expression:

$$(n-1)^2$$

$$\text{Total Operations} = n^2 - 2n + 1$$

Loop E

```
for (int e = 1; e < n; e++) {  
  
    tot = tot + n;  
  
}
```

Arithmetic Expression for Loop E

This loop runs $n - 1$ times (from $e = 1$ to $e = n - 1$). Inside the loop, there is one constant operation: $tot = tot + n$; Which is performed once per iteration. Thus, the loops run $n - 1$ times.

Total Operations for This Section:

Since there is not any nested loop inside Loop E

Thus, the total number of operations is simply the number of iterations of the loop:

$$\text{Total Operations} = n - 1$$

Step 3: Combine all the results

Combine the time complexities for all three sections:

Loop A and Loop B: $(n - 1)^2$

Loop C and Loop D: $(n - 1) + (n - 1)^2$

Loop E: $n - 1$

The final total operations are:

$$\text{Total Operations} = (n - 1)^2 + (n - 1)^2 + (n - 1)$$

Simplify the expression:

$$\begin{aligned} & (n - 1)^2 + (n - 1)^2 + (n - 1) \\ &= n^2 - 2n + 1 + n^2 - 2n + 1 + n - 1 \end{aligned}$$

$$\text{Total Operations} = 2n^2 - 3n + 1$$

Step 4: Big O Analysis

From the arithmetic expression $2n^2 - 3n + 1$ we are able to find the Big O notation by determining the highest-degree term as Big O notation describes the growth rate of the

function as n becomes large. The complexity expression can be decomposed into individual terms:

$2n^2$: This is a quadratic term, as the variable n is squared. The Dominant Term for this term is n^2 . Thus, the Big O Notation for this term is $O(n^2)$.

$3n$: This is a linear term, as the variable n only has a power one. The Dominant Term for this is n . Thus, the Big O Notation for this term is $O(n)$.

1 : This is a constant term, it represents a fixed amount that does not change as n changes.

The Dominant Term for this term is 1. Thus, the Big O Notation for this term is $O(1)$.

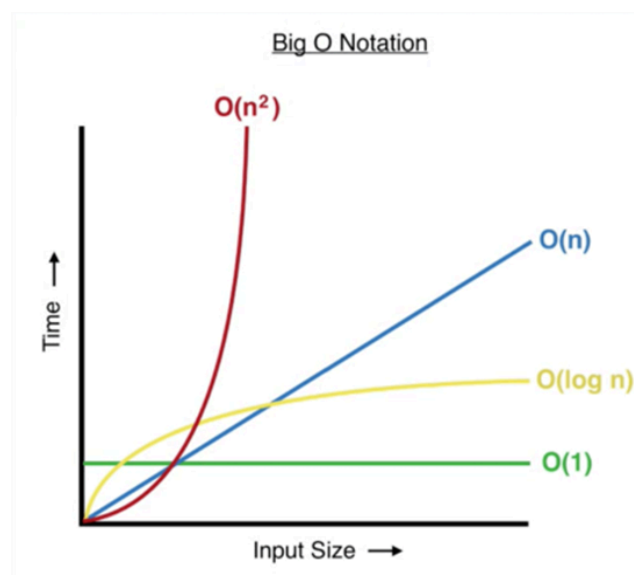


Figure 15 shows the graph of Big O notation

The quadratic term $2n^2$ grows much faster than the linear term $2n$ and the constant term 1 as n increases. In Big O notation, we drop constant factors and non-dominant terms. The constant coefficient 2 in both terms $2n^2$ and $2n$, and the constant 1 are irrelevant, as Big O focuses on the highest degree of n . Thus, the linear term $2n$ and the constant 1 become insignificant for large n when compared to the quadratic term $2n^2$.

Thus,

Overall Dominant term: n^2

Overall Big O complexity: $O(n^2)$

The following is a list of the processing time $T(n)$ spent by an algorithm for solving a problem of size n . For each of these questions (i) and (j), develop a process operation of input n in C++ that matches the complexity given.

i) $n^3 + 2n + n \log_2 n$

Step 1: Determine the Complexity Components

The given complexity expression can be decomposed into individual terms:

n^3 :Cubic term.

$2n$: Linear term.

$n \log_2 n$:Log-linear term.

Step 2: Implement the code

n^3 :Cubic term.

It is a Cubic term. This is achieved by making the loop iterate $n \times n \times n$ times. Inside this loop, you can perform a constant-time operation.

Thus, the code will be:

```
void process(int n) {  
    for (int i = 0; i < n * n * n; ++i) {  
    }  
}
```

Figure 16 shows the code for the term n^3

$2n$: Linear term.

It is a linear term. It simulates linear complexity with a simple loop running twice the number of n . A constant operation is performed inside the loop.

Thus, the code will be:

```
for (int i = 0; i < 2 * n; ++i) {  
}
```

Figure 17 shows the code for the term $2n$

$n \log_2 n$:Log-linear term.

It is a Log-linear term. The function is used to calculate the base-2 logarithm of $n \log_2 n$. This structure creates a time complexity of $n \log_2 n$, because for each of the n iterations, the inner loop runs for $\log_2 n$ iterations.

Thus, the code will be:

```
for (int i = 0; i < n; ++i) {
    int limit = log2(n);
    for (int j = 0; j < limit; ++j) {
    }
}
```

Figure 18 shows the code for the term $n \log_2 n$

Step 3: Combine all the code

Finally, gather all the individual code by each term and will get code following below:

```
#include <iostream>
#include <cmath>
void process(int n) {
    for (int i = 0; i < n * n * n; ++i) {
    }
    for (int i = 0; i < 2 * n; ++i) {
    }
    for (int i = 0; i < n; ++i) {
        int limit = log2(n);
        for (int j = 0; j < limit; ++j) {
        }
    }
}
}
```

Figure 19 shows the final code for the expression given

j) The given problem involves analysing and implementing a process operation that matches a specific time complexity. This task requires breaking down the complexity into its individual components and constructing a corresponding C++ program to simulate the behaviour of each term. Below is the original processing time $T(n)$ spent by an algorithm for solving a problem of size n :

$$2n + 2 + \log_3 n + n^2$$

Step 1: Determine the Complexity Components

The given complexity expression can be decomposed into individual terms:

2n: Linear term.

2: Constant term.

log₃n: Logarithmic term.

n²: Quadratic term.

Step 2: Implement the code

2n: Linear term

It is a linear term. There will be work proportional to $2 \times n$. There is no nested loop as the power of n is only 1. Thus, use a loop that executes $2n$ times.

Thus, the code will be:

```
for (int a = 0; a < 2 * n; a++) {  
    tot = tot + n  
}
```

Figure 20 shows the code for the term $2n$

2: Constant term

It is a constant term. This represents a fixed amount of work that doesn't depend on n . There is no nested loop as there is no n . Ensures a fixed amount of work is done regardless of the input size n . Thus, use a loop that executes twice

The code will be:

```
for (int b = 0; b < 2; b++) {  
    tot = tot + n  
}
```

Figure 21 shows the code for the term 2

log₃n: Logarithm term (base 3)

It is a Logarithmic term and its base 3. The process reduces the input size n by a factor of 3 in each step. There is no nested loop as there is no n in front of the log. Thus, use $n/3$ because it shrinks the input by dividing it into thirds, which aligns with logarithms in base 3.

The code will be:

```
int tot = 0
for (int c = 0; c < n ; c++) {
    n = n/3;
    tot = tot + n
}
```

Figure 22 shows the code for the term $\log_3 n$

n^2 : Quadratic term

It is a quadratic term. There will be one nested loop as the power of n is 2. Thus use two nested loops, each iterating n times, resulting in n^2 total iterations.

The code will be:

```
for (int d = 0; d < n; d++) {
    for (int e = 0; e < n; e++) {
    }
}
```

Figure 23 shows the code for the n^2 term

Step 3: Combine all the code

Finally, gather all the individual code by each term and will get code following below:

```

void processOperation(int n) {
    for (int a = 0; a < 2 * n; a++) {
        tot = tot + n
    }

    for (int b = 0; b < 2; b++) {
        tot = tot + n
    }

    int tot = 0
    for (int c = 0; c < n ; c++) {
        n = n/3;
        tot = tot + n
    }

    for (int d = 0; d < n; d++) {
        for (int e = 0; e < n; e++) {
            }
        }
    }
}

```

Figure 24 shows the final code for the expression given

In conclusion, In this assignment, the time complexity $T(n) = 2n + 2 + \log_3 n + n^2$ was implemented in C++ by simulating its individual components. The process demonstrated how different complexity types such as linear, constant, logarithmic, and quadratic affect the algorithm's performance. In this exercise, time complexity of algorithms and how the sizes of inputs affect the operations was clearly illustrated and serves as a perfect guideline in the design of the algorithms.

MARKING SCHEME

QUESTION PART 1

Items	Marks
Organization	$\frac{1}{2}$
Submission compliance	$\frac{1}{2}$
For question (a) to (f)	
Correct dominant term and Big O – $\frac{1}{2}$ each x 6	3
Justifications – 1 each x 6	6
For question (g & h)	
Correct dominant term and Big O – $\frac{1}{2}$ each x 2	1
Justifications – 2 each x 2	4
For question (i) to (j)	
Correct algorithm – 2 each x 2	4
Comments – $\frac{1}{2}$ each x 2	1
TOTAL	20

QUESTION PART 2

Items	Marks
Problem explanation and group introduction	1
Algorithm visual presentation effectiveness	3
Algorithm visual presentation creativity	2
Algorithm complexity analysis	3
Compliance to limitation (video size/format and duration)	1
Notes: If facial visibility is minimal or the identity of presenter is doubtful (-5 marks)	
TOTAL	10

TOTAL OVERALL 30

Students are expected to participate in group meeting during tutorial. Extra credit of 2 individual marks will be given to in class group meeting participation.

MARKING RUBRICS

For EACH criterion of marks allocated, the following rubrics will be applied:

100% of allocated marks	75% of allocated marks	50% of allocated marks	25% allocated marks	0% allocated marks
• Complete understanding of the problem • A plan that could lead to a correct solution with no algorithmic errors • Correct solution	• Misinterprets minor part of the problem • Substantially correct solution with minor omission or procedural error	• Misinterprets major part of the problem • Partially correct solution but with major fault • Computational error, partial solution for problem.	• Completely misinterprets the problem • Substantially inappropriate solution	• No attempt • No answer or wrong answer based upon an inappropriate solution

END OF MARKING SCHEME SECTION